

Documentation Officielle du Langage MacLaire

Spécifications Techniques et Syntaxe

1. Introduction

Le MacLaire est un langage de programmation impératif et modulaire, conçu pour une lisibilité maximale proche du langage naturel tout en offrant un contrôle strict sur la gestion des ressources et de la mémoire. Il se distingue par son système d'indexation de fichiers et sa syntaxe de contrôle de flux unique.

2. Structure du Programme

2.1 Initialisation

Tout script MacLaire doit commencer par l'initialisation de son instance propre pour garantir la persistance des variables et l'exécution du flux principal.

```
[source: 1] execute {self}
```

2.2 Commentaires

Les commentaires sont délimités par des crochets `[]` et peuvent être insérés n'importe où dans le code, même au milieu d'une instruction.

```
existe a [déclaration de la variable a]
```

3. Gestion des Fichiers (Invocations)

Le MacLaire utilise un système de mapping pour lier des fichiers externes à des index numériques.

```
invoque nom_du_fichier at {index}
```

L'exécution d'un fichier lié se fait via la commande `execute {index}`.

4. Variables et Persistance

4.1 Déclaration et Affectation

Commande	Description
<code>existe nom</code>	Déclare une variable en mémoire.
<code>give to</code>	Assigne une valeur ou un résultat à une variable.

```
existe resultat  
resultat give to a + b
```

4.2 Cycle de vie de la variable

- `variable$crush <var>` : Détruit la variable et libère la mémoire.
- `variable$empty <var>` : Vide le contenu de la variable.
- `variable$state <var>` : Retourne l'état ou la valeur actuelle.
- `variable$instancelimit <var> x mo` : Limite la taille mémoire de la variable.

4.3 Portée (Heritage)

Par défaut, les variables sont globales. Pour restreindre une variable à des modules spécifiques :

```
nom_variable exist for {1} and {2}
```

5. Entrées et Sorties

5.1 Affichage

- `imprimate <"texte">` : Affiche une chaîne de caractères.
- `imprimate$variable <nom>` : Affiche la valeur d'une variable.
- `imprimate$multiline <"...">` : Affiche un bloc de texte sur plusieurs lignes.

5.2 Saisie (Input)

La saisie est typée pour éviter les erreurs de compilation :

```
choise < variable / int > with message <"Texte"> give to nom_variable
```

6. Structures de Contrôle

6.1 Conditions (Check)

Le MacLaire utilise une structure de vérification en cascade :

```
check [condition] do /  
  [bloc d'actions]  
/  
no check but check [condition] do /  
  [bloc d'actions]  
/  
absolute no check do /  
  [bloc par défaut]  
/
```

6.2 Boucles

Boucle Pour : `exponentiate value a from 1 to 10 doing: / [bloc] /`

Boucle Tant que : Basée sur les états `onTrue` ou `onFalse` .

```
onTrue True do : / [s'exécute tant que vrai] /
```

7. Gestion des Erreurs

Le langage possède des messages d'erreurs natifs explicites :

- **Erreur de type :** `variable non gratas` .
- **Division par zéro :** `impossible operation the computeur did cannot do operation "operation"` .

Note : Les blocs d'actions sont délimités par des slashes `/ /` et peuvent englober plusieurs lignes pour une meilleure organisation structurelle.